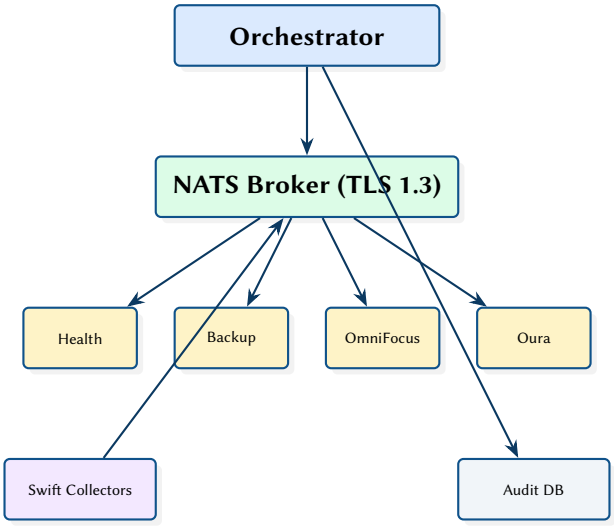
Secure Multi-Agent Personal Assistant

Zero-Trust Agent Orchestration with NATS, Container Isolation, and Capability-Gated Execution

Simon Knight

March 2026 • Version 0.1.0



Abstract. This report presents a security-first multi-agent system where 40+ specialised agents run in isolated Docker containers, coordinated by a central orchestrator over a hardened NATS message broker with TLS 1.3 and per-subject ACLs. The core design principle is strict isolation: compromise of one agent must not cascade to other agents or to the orchestrator. The system implements defence-in-depth through capability tokens (signed, expiring, nonce-replay-protected), seccomp deny-by-default profiles, read-only filesystems, and an append-style SQLite audit trail. macOS host collectors written in Swift provide OS-only integrations (HealthKit, EventKit) over NATS, while the orchestrator uses an LLM for DAG-based workflow planning.

Version	Date	Changes
0.1.0	March 2026	Initial architecture report

Contents

List of Figures

List of Tables

List of Design Decisions & Rules

1 Introduction and Motivation

Personal assistant systems that integrate health data, calendars, task management, and home automation require access to sensitive information across multiple domains. A monolithic architecture creates a single point of compromise; if any component is exploited, all data is exposed.

This system adopts a zero-trust architecture: agents are treated as potentially compromised, and the system relies on explicit boundaries (network, capabilities, filesystem, audit) rather than trusting agent code.

: Zero-Trust Agent Model

Every agent runs in an isolated container with no network access except to NATS. Each task dispatch includes a short-lived capability token that agents must validate (signature, expiry, nonce replay prevention) before acting. Agents cannot communicate with each other directly.

2 System Architecture

2.1 Message Broker

NATS [1] provides the messaging backbone with:

- **TLS 1.3** for all connections
- **NKEY authentication** using Ed25519 keypairs per agent
- **Subject-level ACLs** restricting each agent to its own namespace (e.g. `agent.health.*`)
- **JetStream** for durable message delivery

2.2 Orchestrator

The Python orchestrator receives user requests and plans execution as a Directed Acyclic Graph (DAG) of tasks using an Large Language Model (LLM). For each task, it:

1. Issues a short-lived capability token (signed, 60s expiry)
2. Dispatches the task to the appropriate agent via NATS
3. Collects results and routes to downstream tasks
4. Emits audit events and OpenTelemetry [2] spans

Design Decision 1: LLM-Based Task Planning

The orchestrator uses an LLM to decompose user requests into task DAGs rather than hard-coding workflow definitions. This enables natural language requests (“prepare my morning briefing”) to dynamically compose health, calendar, weather, and task agents based on the request context.

2.3 Container Hardening

Each agent container enforces defence-in-depth [3]:

Table 1. Container security controls.

Control	Implementation
Seccomp	Deny-by-default profile; only allowed syscalls whitelisted
Read-only FS	`read_only: true` on container root
No new privs	`no-new-privileges` security option
Network	Only NATS port exposed; no outbound internet
Capabilities	All Linux capabilities dropped

2.4 Swift Collectors

macOS-only integrations (HealthKit, EventKit) run as native Swift binaries on the host, publishing data to NATS over TLS. These collectors have no agent logic; they simply transform OS data into NATS messages for consumption by the appropriate agent container.

3 Capability Token System

Insight: Running HealthKit and EventKit access as host-side Swift binaries rather than in containers avoids the need to grant Docker containers access to macOS frameworks, which would require disabling sandbox protections entirely.

Each task dispatch includes a JWT-like capability token:

```
@dataclass
class CapabilityToken:
    task_id: str
    agent_id: str
    permissions: list[str]   # e.g. ["read:health", "write:summary"]
    issued_at: datetime
    expires_at: datetime     # 60-second TTL
    nonce: str               # UUID for replay prevention
    signature: bytes         # Ed25519 signature
```

Agents validate all fields before executing any task. A nonce cache (bounded LRU) prevents replay attacks within the token's lifetime.

: Minimum Privilege Tokens

Each token grants only the permissions required for that specific task. A health summary task receives `read:health` but not `write:health`. Token permissions are determined by the orchestrator's task planner based on the task type definition.

4 Audit Trail

An append-style SQLite database (WAL mode) records all system events: task dispatches, agent responses, capability token issuance and validation, security violations, and orchestrator decisions. The audit trail is write-only from the perspective of agents; only the orchestrator can read it for dashboarding.

5 Observability

Structured logging and OpenTelemetry tracing provide end-to-end visibility. Each task dispatch creates a trace span that propagates through NATS to the agent, enabling distributed tracing across the orchestrator–broker–agent chain. Traces are collected by Jaeger for visualisation.

6 Design Summary

Table 2. Key design decisions.

Decision	Rationale
Zero-trust agents	Compromise containment; each agent is a blast radius
NATS with NKEY	Per-agent identity; subject ACLs enforce namespace isolation
Short-lived tokens	60s TTL limits replay window; nonce cache prevents reuse
Seccomp deny-default	Minimises kernel attack surface per container
Swift host collectors	macOS framework access without container sandbox weakening
LLM task planning	Dynamic workflow composition from natural language

References

[1] Synadia, *NATS: Cloud native messaging system*, 2024. [Online]. Available: https://nats.io

[2] CNCF, *OpenTelemetry: Observability framework*, 2024. [Online]. Available: https://opentelemetry.io

[3] Linux Kernel, *Seccomp: Secure computing mode*, 2024. [Online]. Available: https://www.kernel.org/doc/html/latest/userspace-api/seccomp_filter.html